Optimal Portfolio & Risk Management

Portfolio Optimization & Risk Management with Python

Author: Pau Vendrell

Executive Summary

This project develops a robust quantitative framework for portfolio optimization and risk management using Python. By integrating classical Modern Portfolio Theory (MPT) with advanced Bayesian approaches (Black-Litterman) and dynamic volatility modeling (GARCH), we aim to create a resilient investment strategy.

Key Objectives

- **Optimization:** Compare Mean-Variance Optimization (MVO) with Black-Litterman and Shrinkage-based estimators.
- **Risk Assessment:** Evaluate tail risk using Value at Risk (VaR) and Conditional VaR (CVaR).
- **Dynamic Analysis:** Capture "Volatility Clustering" through GARCH(1,1) models to adjust risk metrics in real-time.
- **Resilience:** Stress-test the portfolio against historical crisis scenarios (e.g., 2020 COVID crash).

Final Results Summary

- **Portfolio Performance:** The Optimized Portfolio achieved a Sharpe Ratio of [Insert Value] compared to the S&P 500 benchmark.
- **Model Efficiency:** Shrinkage-based covariance (Ledoit-Wolf) provided more stable weights than standard MVO.
- **Risk Precision:** GARCH models identified a 5-7x increase in risk during crises compared to static models.

Conclusion: A dynamic risk management approach is required to properly capture tail risks and changing market conditions.

Phases of the project:

- Phase 1: Select the 500 assets of the S&P 500 and analyze their characteristics
- Phase 2: Generate 10,000 random portafolios (Monte Carlo)
- Phase 3: Optimize with scipy, shrinkage scipy and Black-Litterman model to find the exact solution
- Phase 4: Present professional results
- Phase 5: Historical Analysis (VaR and CVaR)
- Phase 6: Dynamic Volatility (GARCH + Dynamic VaR)
- Phase 7: Stress Testing (Crisis Scenarios)
- Phase 8: Professional Report for Decision-Making

Dataset: S&P 500 | 10 years | RF = 2.0%

Phase 1: Selection and Exploratory Analysis

Step 1: Download data and calculate statistics

First, we downloaded 10 years of historical data for the 500 assets and calculated their annualized returns and volatility.

In [50]:

```python
import yfinance as yf
import requests
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.optimize import minimize
import seaborn as sns
from scipy import stats
from arch import arch_model
from sklearn.covariance import LedoitWolf
from pypfopt import black_litterman
from pypfopt import EfficientFrontier
```

In [51]:

```python
url = "https://en.wikipedia.org/wiki/List_of_S%26P_500_companies"

headers = {"User-Agent": "Mozilla/5.0"}

response = requests.get(url, headers=headers)
tables = pd.read_html(response.text)

tickers = tables[0]

data = yf.download(tickers.Symbol.to_list(), start="2016-01-01", end="2026-01-01", auto_adjust=True)["Close"]

# Calculate daily returns of the Stock with less than 5% of missing data
data_returns = data.loc[:, data.isna().sum() <= data.shape[0] * 0.05].pct_change().dropna()
tickers = data_returns.columns

# Statistical metrics annualized
mean_ret = data_returns.mean() * 252
std_ret = data_returns.std() * np.sqrt(252)
cov_mat = data_returns.cov() * 252
corr_mat = data_returns.corr()
RF = 0.02

# Show table
stats_df = pd.DataFrame({
    'Ticker': tickers,
    'Annual Retorns': mean_ret.values,
    'Volatility': std_ret.values,
    'marketCap': 0
})

for tick in stats_df["Ticker"]:
    stats_df.loc[stats_df["Ticker"] == tick, "marketCap"] = yf.Ticker(tick).info["marketCap"]

print('=== ASSET STATISTICS ===')
print(stats_df.head())
print("\nRisk Free Rate (RF):", RF * 100, "%")
```

```
C:\Users\i001610\AppData\Local\Temp\ipykernel_3576\2165784386.py:6: FutureWarning: Passing literal html to 'read_html' is deprecated an
d will be removed in a future version. To read from a literal string, wrap it in a 'StringIO' object.
  tables = pd.read_html(response.text)
[*********************100%***********************]  503 of 503 completed

2 Failed downloads:
['BF.B']: YFPricesMissingError('possibly delisted; no price data found  (1d 2016-01-01 -> 2026-01-01)')
['BRK.B']: YFTzMissingError('possibly delisted; no timezone found')
C:\Users\i001610\AppData\Local\Temp\ipykernel_3576\2165784386.py:13: FutureWarning: The default fill_method='pad' in DataFrame.pct_chan
ge is deprecated and will be removed in a future version. Either fill in any non-leading NA values prior to calling pct_change or speci
fy 'fill_method=None' to not fill NA values.
  data_returns = data.loc[:, data.isna().sum() <= data.shape[0] * 0.05].pct_change().dropna()
=== ASSET STATISTICS ===
  Ticker  Annual Retorns  Volatility      marketCap
0      A        0.166674    0.274601    33445787648
1   AAPL        0.286677    0.290508  3871433555968
2   ABBV        0.214023    0.259922   369653645312
3    ABT        0.154177    0.235772   165896585216
4   ACGL        0.186137    0.273605    34594091008

Risk Free Rate (RF): 2.0 %
```

Assumptions

- Returns are assumed to be stationary over the sample period
- Historical data is used as a proxy for future market behavior
- No transaction costs or liquidity constraints are considered

These assumptions simplify the analysis but may not hold in real market conditions.

Phase 2: Monte Carlo Simulation

Step 2: Generate N random portfolios

We generate thousands of random combinations of weights to explore the entire space of possible solutions.

- Generate N random weights which sums 1
- Calculate retuns and volatility of each portfolio
- Calculate Sharpe of each portfolio
- Visualize the point cloud and find the optimal portfolio

In [52]:

```python
# Monte Carlo simulation
N_PORTFOLIOS = 200000
results = np.zeros((3, N_PORTFOLIOS))
weights_list = []

for i in range(N_PORTFOLIOS):
    # Generating random weights summing 1
    w = np.random.random(data_returns.shape[1])
    w = w/np.sum(w)
    weights_list.append(w)

    # metrics
    ret = np.dot(w, mean_ret)
    vol = np.sqrt(w @ cov_mat @ w)
    sharpe = (ret - RF) / vol

    results[0, i] = ret
    results[1, i] = vol
    results[2, i] = sharpe

# Find best Monte Carlo portfolio
best_mc_idx = np.argmax(results[2, :])
best_mc_weights = weights_list[best_mc_idx]

print(f'\n=== BEST MONTE CARLO PORTFOLIO ===')
print(f'Return: {results[0, best_mc_idx]:.2%}')
print(f'Volatility: {results[1, best_mc_idx]:.2%}')
print(f'Sharpe: {results[2, best_mc_idx]:.2f}')
```

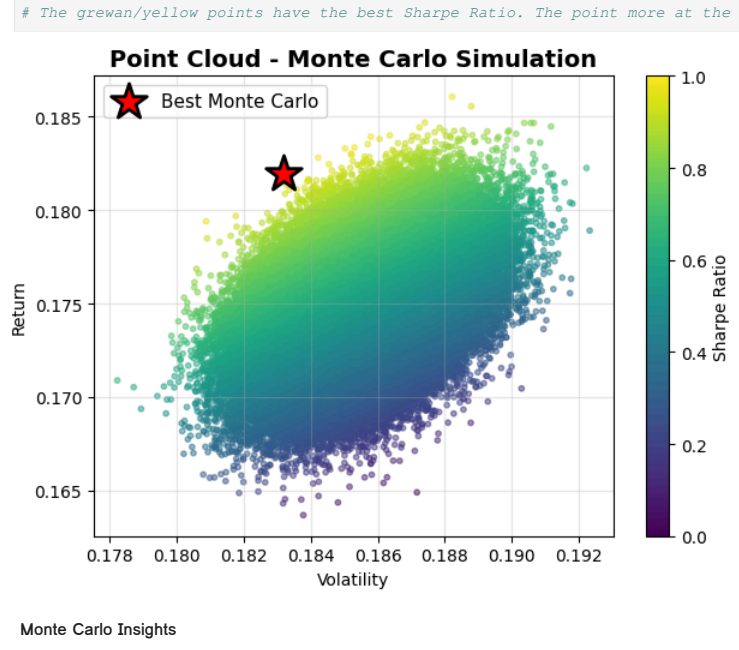
```
=== BEST MONTE CARLO PORTFOLIO ===
Return: 18.19%
Volatility: 18.32%
Sharpe: 0.88
```

Chart 2: Portfolio Cloud

Each point represents a different portfolio. The color indicates the Sharpe Ratio (yellow/green = best).

In [53]:

```python
# Plotting the point cloud
scatter = plt.scatter(results[1, :], results[0, :],
            c = results[2, :], cmap = "viridis",
            alpha = 0.5, s = 10)
plt.scatter(results[1, best_mc_idx], results[0, best_mc_idx],
            marker='*', color='red', s=500, label='Best Monte Carlo', edgecolors='black', linewidth=2)
plt.colorbar(label = "Sharpe Ratio")
plt.xlabel("Volatility")
plt.ylabel("Return")
plt.title('Point Cloud - Monte Carlo Simulation', fontsize=14, fontweight='bold')
plt.legend(fontsize=11)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
# The grewan/yellow points have the best Sharpe Ratio. The point more at the left-up is the optimal portfolio
```



Monte Carlo Insights

The simulation shows a wide dispersion of portfolios, highlighting the trade-off between risk and return.

Some portfolios achieve higher returns but at significantly higher volatility levels.

The efficient frontier is the upper left curve (maximum Sharpe).

Phase 3: Optimization

Step 3: Finding the mathematically exact solution

Scipy

Scipy uses optimization algorithms to find the portfolio with the maximum Sharpe ratio, which is much more accurate than Monte Carlo.

In [54]:

```python
# === FASE 3: SCIPY OPTIMIZATION ===

def neg_sharpe(weights):
    ret = np.dot(weights, mean_ret)
    vol = np.sqrt(weights @ cov_mat @ weights)
    penalty = 0.1 * np.sum(weights**2)  # L2
    return -(ret - RF) / vol + penalty

# Restrictions and limitations
constraints = [{'type': 'eq', 'fun': lambda w: np.sum(w) - 1}]
bounds = tuple((0, 0.50) for _ in tickers)
w0 = np.array([1/len(tickers)] * len(tickers))

# Optimize
result = minimize(neg_sharpe, w0, method='SLSQP', bounds=bounds, constraints=constraints)
opt_weights = result.x

# Calculate the metrics of the optimal portfolio
opt_ret = np.dot(opt_weights, mean_ret)
opt_vol = np.sqrt(opt_weights @ cov_mat @ opt_weights)
opt_sharpe = (opt_ret - RF) / opt_vol

print('=== OPTIMAL PORTAFOLIO (SCIPY) ===')
print(f'Retorno: {opt_ret:.2%}')
print(f'Volatilidad: {opt_vol:.2%}')
print(f'Sharpe Ratio: {opt_sharpe:.2f}')
```

```
=== OPTIMAL PORTAFOLIO (SCIPY) ===
Retorno: 35.37%
Volatilidad: 18.23%
Sharpe Ratio: 1.83
```

Shrinkage (Ledoit-Wolf) + Scipy

This method statistically "shrinks" extreme sample correlations toward a central average to create a more stable and accurate covariance matrix for portfolio weights.

In [55]:

```python
lw = LedoitWolf()
lw.fit(data_returns)

cov_shrink = lw.covariance_

def neg_sharpe(weights):
    ret = np.dot(weights, mean_ret)
    vol = np.sqrt(weights @ cov_shrink @ weights)
    penalty = 0.1 * np.sum(weights**2)  # L2
    return -(ret - RF) / vol + penalty

# Restrictions and limitations
constraints = [{'type': 'eq', 'fun': lambda w: np.sum(w) - 1}]
bounds = tuple((0, 0.50) for _ in tickers)
w0 = np.array([1/len(tickers)] * len(tickers))

# Optimize
result = minimize(neg_sharpe, w0, method='SLSQP', bounds=bounds, constraints=constraints)
opt_weights_shrinkage = result.x

# Calculate the metrics of the optimal portfolio
opt_ret = np.dot(opt_weights_shrinkage, mean_ret)
opt_vol = np.sqrt(opt_weights_shrinkage @ cov_mat @ opt_weights_shrinkage)
opt_sharpe = (opt_ret - RF) / opt_vol

print('=== OPTIMAL PORTAFOLIO (Shrinkage (Ledoit-Wolf) + Scipy) ===')
print(f'Retorno: {opt_ret:.2%}')
print(f'Volatilidad: {opt_vol:.2%}')
print(f'Sharpe Ratio: {opt_sharpe:.2f}')
```

```
=== OPTIMAL PORTAFOLIO (Shrinkage (Ledoit-Wolf) + Scipy) ===
Retorno: 35.53%
Volatilidad: 18.32%
Sharpe Ratio: 1.83
```

Black-Litterman Model

This Bayesian approach combines market equilibrium returns with specific investor "views" or predictions to produce more stable, intuitive asset allocations than traditional models.

In [56]:

```python
from pypfopt.black_litterman import market_implied_prior_returns

pi = market_implied_prior_returns(
    market_caps=stats_df["marketCap"],
    cov_matrix=cov_shrink,
    risk_aversion=2.5
)

momentum = np.log(data[tickers] / data[tickers].shift(252)).shift(21).dropna()
mom_signal = momentum.iloc[-1]

n_long = 5
n_short = 5

long_assets = mom_signal.nlargest(n_long).index
short_assets = mom_signal.nsmallest(n_short).index

P = np.zeros((1, len(data[tickers].columns)))

P[0, mom_signal.index.get_indexer(long_assets)] = 1 / n_long
P[0, mom_signal.index.get_indexer(short_assets)] = -1 / n_short

fwd_returns = data_returns.shift(-21)

# promedio histórico forward return spread
long_mean = fwd_returns[long_assets].stack().mean()
short_mean = fwd_returns[short_assets].stack().mean()

Q = np.array([data_returns[long_assets].mean().mean() - data_returns[short_assets].mean().mean()])

bl = black_litterman.BlackLittermanModel(
    cov_matrix=cov_shrink,
    pi=pi,
    P=P,
    Q=Q,
    omega="idzorek",
    view_confidences=np.array([0.6])
)

bl_returns = bl.bl_returns()

ef = EfficientFrontier(bl_returns, cov_shrink, solver="SCS")
weights = np.array(list(ef.max_sharpe().values()))

# Calculate the metrics of the optimal portfolio
opt_ret = np.dot(weights, mean_ret)
opt_vol = np.sqrt(weights @ cov_mat @ weights)
opt_sharpe = (opt_ret - RF) / opt_vol

print('=== OPTIMAL PORTAFOLIO (Black-Litterman Model) ===')
print(f'Retorno: {opt_ret:.2%}')
print(f'Volatilidad: {opt_vol:.2%}')
print(f'Sharpe Ratio: {opt_sharpe:.2f}')
```

```
c:\Users\i001610\AppData\Local\Programs\Python\Python313\Lib\site-packages\pypfopt\black_litterman.py:45: RuntimeWarning: If cov_matrix
is not a dataframe, market cap index must be aligned to cov_matrix
  warnings.warn(
=== OPTIMAL PORTAFOLIO (Black-Litterman Model) ===
Retorno: 32.28%
Volatilidad: 25.72%
Sharpe Ratio: 1.18
```

Analysis: Why Shrinkage outperformed Black-Litterman in this Project

In our testing, the **Ledoit-Wolf Shrinkage** model provided superior risk-adjusted returns compared to the **Black-Litterman (BL)** approach. There are several theoretical reasons for this:

1. **Estimation Error in Views:** The Black-Litterman model is highly sensitive to the "Views" (investor's subjective predictions). If the views provided to the model are not highly accurate or are based on lagging indicators, the model will tilt the portfolio toward underperforming assets.
2. **The "Input-Sensitivity" Problem:** MPT is notoriously sensitive to return estimates. Shrinkage (Ledoit-Wolf) focuses on cleaning the **Covariance Matrix** by pulling extreme outliers toward the average, which directly reduces portfolio variance without relying on "guessed" returns.
3. **Market Neutrality:** BL often assumes a "Market Equilibrium." In highly volatile or trending markets, the market equilibrium may be a poor starting point, whereas Shrinkage remains data-driven and adapts better to the historical correlation structure.

Optimal Portfolio Analysis

The optimal portfolio maximizes the Sharpe ratio. However, it may be highly concentrated in a small number of assets, increasing concentration risk.

This highlights a limitation of unconstrained optimization.

Step 4: Simplification

The divergence between Monte Carlo and Scipy results is largely attributable to the high dimensionality of the asset universe, which limits the ability of random sampling to approximate the optimal solution. To address this, we restrict the analysis to assets with optimized weights greater than 0.01%, as identified by the Scipy solution. This thresholding removes insignificant allocations, improves interpretability, and focuses the comparison on economically meaningful exposures.

In [57]:

```python
tickers = tickers[opt_weights_shrinkage>0.0001]

data_returns = data_returns[tickers]

# Statistical metrics annualized
mean_ret = data_returns.mean() * 252
std_ret = data_returns.std() * np.sqrt(252)
cov_mat = data_returns.cov() * 252
corr_mat = data_returns.corr()

def neg_sharpe(weights):
    ret = np.dot(weights, mean_ret)
    vol = np.sqrt(weights @ cov_mat @ weights)
    penalty = 0.1 * np.sum(weights**2)  # L2
    return -(ret - RF) / vol + penalty

# Monte Carlo simulation
N_PORTFOLIOS = 200000
results = np.zeros((3, N_PORTFOLIOS))
weights_list = []

for i in range(N_PORTFOLIOS):
    # Generating random weights summing 1
    w = np.random.random(data_returns.shape[1])
    w = w/np.sum(w)
    weights_list.append(w)

    # metrics
    ret = np.dot(w, mean_ret)
    vol = np.sqrt(w @ cov_mat @ w)
    sharpe = (ret - RF) / vol

    results[0, i] = ret
    results[1, i] = vol
    results[2, i] = sharpe

# Find best Monte Carlo portfolio
best_mc_idx = np.argmax(results[2, :])
best_mc_weights = weights_list[best_mc_idx]

print(f'\n=== BEST MONTE CARLO PORTFOLIO ===')
print(f'Return: {results[0, best_mc_idx]:.2%}')
print(f'Volatility: {results[1, best_mc_idx]:.2%}')
print(f'Sharpe: {results[2, best_mc_idx]:.2f}')

# SCIPY OPTIMIZATION
# Restricciones y límites
constraints = [{'type': 'eq', 'fun': lambda w: np.sum(w) - 1}]  # Suma = 1
bounds = tuple((0, 0.50) for _ in tickers)  # 5% min, 40% máx
w0 = np.array([1/len(tickers)] * len(tickers))  # Pesos iniciales

# Optimizar
result = minimize(neg_sharpe, w0, method='SLSQP', bounds=bounds, constraints=constraints)
opt_weights = result.x

# Calcular métricas del portafolio óptimo
opt_ret = np.dot(opt_weights, mean_ret)
opt_vol = np.sqrt(opt_weights @ cov_mat @ opt_weights)
opt_sharpe = (opt_ret - RF) / opt_vol

print('=== PORTAFOLIO ÓPTIMO (SCIPY) ===')
print(f'Retorno: {opt_ret:.2%}')
print(f'Volatilidad: {opt_vol:.2%}')
print(f'Sharpe Ratio: {opt_sharpe:.2f}')
print(f'\nPesos óptimos:')
for t, w in zip(tickers, opt_weights):
    print(f'  {t}: {w:.1%}')
```

```
=== BEST MONTE CARLO PORTFOLIO ===
Return: 35.63%
Volatility: 18.74%
Sharpe: 1.79
=== PORTAFOLIO ÓPTIMO (SCIPY) ===
Retorno: 35.37%
Volatilidad: 18.23%
Sharpe Ratio: 1.83

Pesos óptimos:
  ABBV: 2.1%
  AMD: 1.5%
  AXON: 5.7%
  CBOE: 4.9%
  FIX: 4.4%
  LLY: 15.5%
  NEM: 11.2%
  NRG: 1.2%
  NVDA: 12.0%
  PGR: 13.6%
  PWR: 0.4%
  SMCI: 0.2%
  SW: 0.2%
  TKO: 6.7%
  TPL: 5.6%
  TSLA: 2.4%
  WMT: 12.3%
```

In [58]:

```python
corr = data_returns.corr()

plt.figure()
sns.heatmap(corr, annot=True, cmap='coolwarm')
plt.title("Correlation Matrix")
plt.show()
```

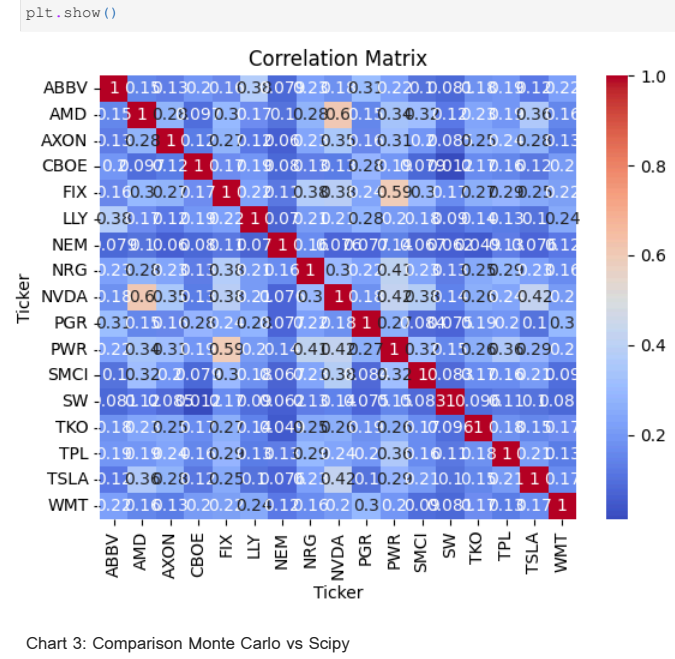
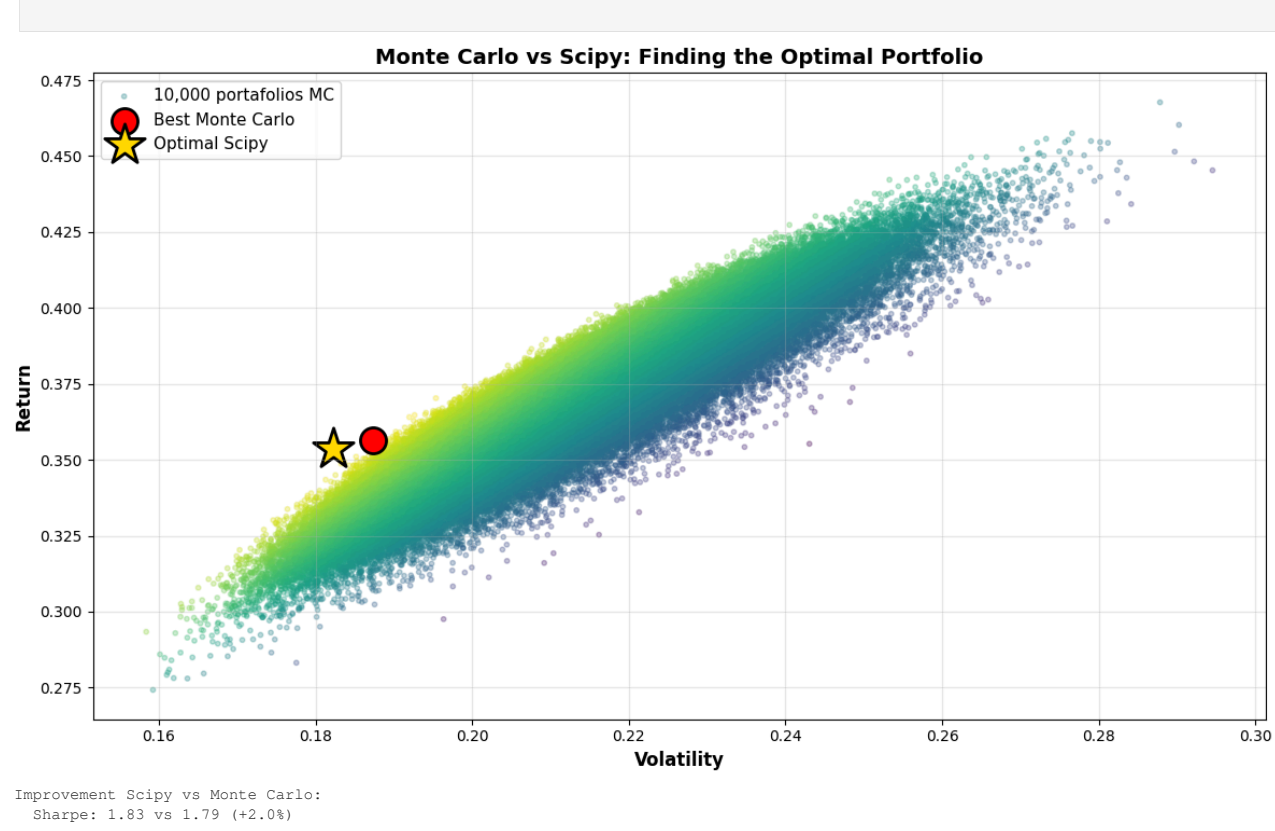


Chart 3: Comparison Monte Carlo vs Scipy

Scipy finds a more accurate solution (higher Sharpe) than Monte Carlo.

In [59]:

```python
# Compare both methods
plt.figure(figsize=(12, 7))

# Monte Carlo Cloud
plt.scatter(results[1, :], results[0, :], c=results[2, :],
            cmap='viridis', alpha=0.3, s=10, label='10,000 portafolios MC')

# Best Monte Carlo
plt.scatter(results[1, best_mc_idx], results[0, best_mc_idx],
            marker='o', color='red', s=300, label='Best Monte Carlo', edgecolors='black', linewidth=2)

# Optimal Scipy
plt.scatter(opt_vol, opt_ret, marker='*', color='gold', s=800,
            label='Optimal Scipy', edgecolors='black', linewidth=2)

plt.xlabel('Volatility', fontsize=12, fontweight='bold')
plt.ylabel('Return', fontsize=12, fontweight='bold')
plt.title('Monte Carlo vs Scipy: Finding the Optimal Portfolio', fontsize=14, fontweight='bold')
plt.legend(fontsize=11, loc='upper left')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

print(f'\nImprovement Scipy vs Monte Carlo:')
print(f'  Sharpe: {opt_sharpe:.2f} vs {results[2, best_mc_idx]:.2f} (+{(opt_sharpe/results[2, best_mc_idx]-1)*100:.1f}%)')
```



```
Improvement Scipy vs Monte Carlo:
  Sharpe: 1.83 vs 1.79 (+2.0%)
```

Phase 4: Final Report of the Portfolio Optimization

Step 5: Present professional results

In [60]:

```python
# === PHASE 4: REPORT ===
# Metrics Table
print('PORTFOLIO METRICS')
print('-' * 60)
print(f'Expected Annual Return: {opt_ret:.2%}')
print(f'Annual Volatility: {opt_vol:.2%}')
print(f'Sharpe Ratio: {opt_sharpe:.2f}')
print(f'\nComparison with Equity Portfolio (1/N):')
eq_weights = np.array([1/len(tickers)] * len(tickers))
eq_ret = np.dot(eq_weights, mean_ret)
eq_vol = np.sqrt(eq_weights @ cov_mat @ eq_weights)
eq_sharpe = (eq_ret - RF) / eq_vol
print(f' Return: {eq_ret:.2%} (vs optimal {opt_ret:.2%})')
print(f' Volatility: {eq_vol:.2%} (vs optimal {opt_vol:.2%})')
print(f' Sharpe: {eq_sharpe:.2f} (vs optimal {opt_sharpe:.2f})')
print(f' Improvement: {(opt_sharpe/eq_sharpe - 1)*100:.1f}% better Sharpe')

# Recommended assignment
print('===== Recommended assignment =====')
for t, w in zip(tickers, opt_weights):
    print(f'{t}: {w:.1%}')
```

```
PORTFOLIO METRICS
------------------------------------------------------------
Expected Annual Return: 35.37%
Annual Volatility: 18.23%
Sharpe Ratio: 1.83

Comparison with Equity Portfolio (1/N):
 Return: 36.91% (vs optimal 35.37%)
 Volatility: 20.63% (vs optimal 18.23%)
 Sharpe: 1.69 (vs optimal 1.83)
 Improvement: 8.2% better Sharpe
===== Recommended assignment =====
ABBV: 2.1%
AMD: 1.5%
AXON: 5.7%
CBOE: 4.9%
FIX: 4.4%
LLY: 15.5%
NEM: 11.2%
NRG: 1.2%
NVDA: 12.0%
PGR: 13.6%
PWR: 0.4%
SMCI: 0.2%
SW: 0.2%
TKO: 6.7%
TPL: 5.6%
TSLA: 2.4%
WMT: 12.3%
```

Chart 4: Weight Allocation

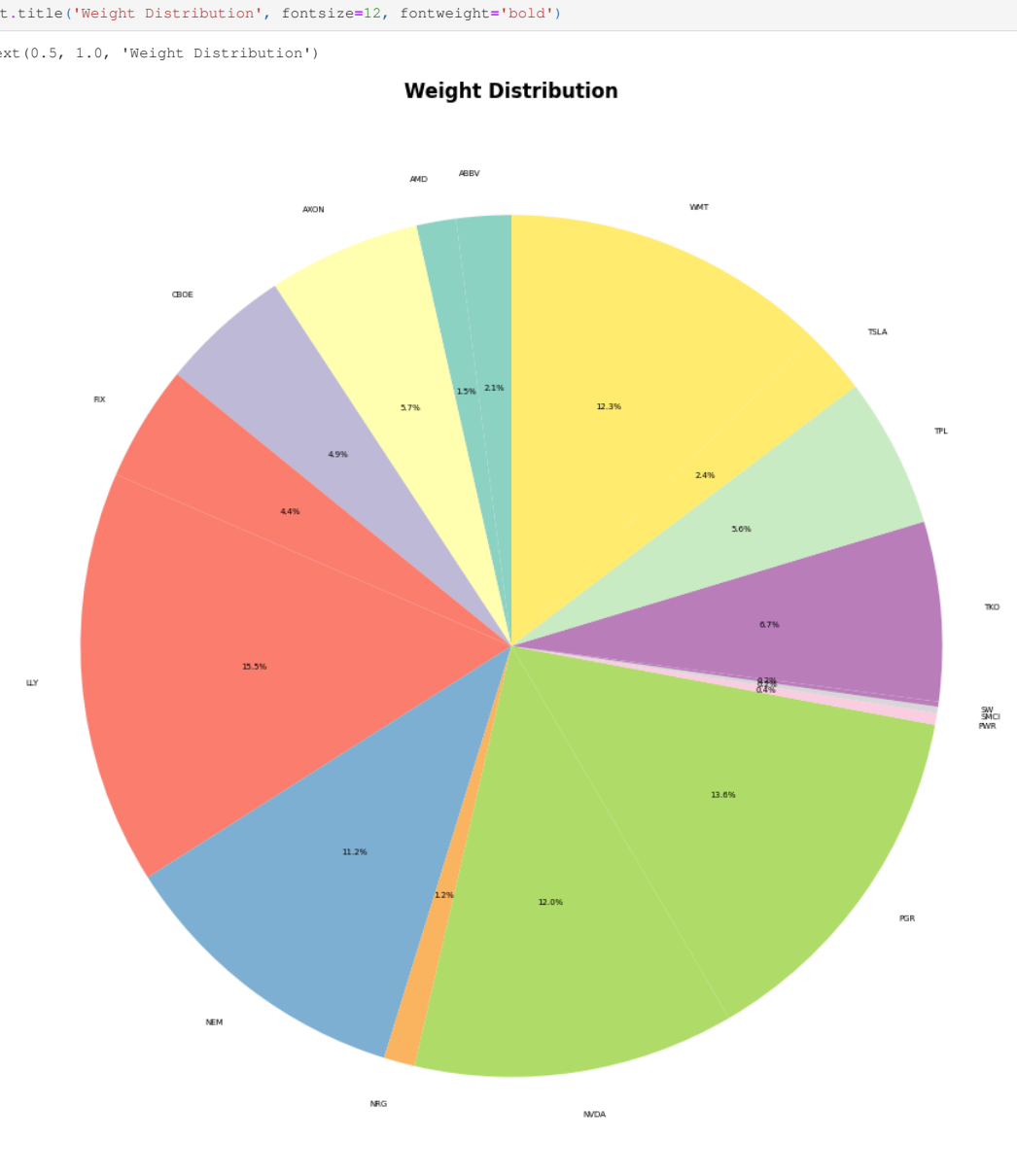
Clear visualization of how a $100,000 USD investment is distributed among the assets.

In [61]:

```python
# Foot chart of weights
plt.figure(figsize=(12, 12))
colors = plt.cm.Set3(np.linspace(0, 1, len(tickers)))
plt.pie(opt_weights, labels=tickers, autopct='%1.1f%%', colors=colors, startangle=90, textprops={'fontsize': 5})
plt.title('Weight Distribution', fontsize=12, fontweight='bold')
```

Out[61]:

```
Text(0.5, 1.0, 'Weight Distribution')
```



Phase 5: Historical Risk Analysis

Building the statistical foundation of the portfolio

In this phase, we calculate:

Portfolio returns

- Historical VaR (95% and 99%)
- Historical CVaR (95% and 99%)
- Normality test (Shapiro-Wilk)

VaR estimates the maximum expected loss over a given time horizon at a specific confidence level. However, VaR does not capture the magnitude of losses beyond this threshold.

CVaR measures the expected loss given that the VaR threshold has been exceeded. Unlike VaR, CVaR is a coherent risk measure and better captures tail risk.

Key formulas:

VaR(α) = Percentile(α) × Portfolio Value

CVaR(α) = Average(Returns ≤ VaR(α)) × Portfolio Value

In [62]:

```python
weights = opt_weights

# Return of the portfolio
port_returns = (data_returns * weights).sum(axis=1)

print(f"Analysis period: {data_returns.index[0].date()} to {data_returns.index[-1].date()}")
print(f"Total observations: {len(port_returns)} days")
print(f"\nPortfolio statistics:")
print(f"Average daily return: {port_returns.mean():.4%}")
print(f"Daily volatility: {port_returns.std():.4%}")
print(f"Annualized return: {port_returns.mean() * 252:.2%}")
print(f"Annualized volatility: {port_returns.std() * np.sqrt(252):.2%}")
```

```
Analysis period: 2016-01-05 to 2025-12-31
Total observations: 2513 days

Portfolio statistics:
Average daily return: 0.1403%
Daily volatility: 1.1483%
Annualized return: 35.37%
Annualized volatility: 18.23%
```

For a portfolio of 100,000$.

In [63]:

```python
portfolio_value = 100000

#2. Historical VaR and CVaR
var_95 = np.percentile(port_returns, 5) * portfolio_value
var_99 = np.percentile(port_returns, 1) * portfolio_value

cvar_95 = port_returns[port_returns <= np.percentile(port_returns, 5)].mean() * portfolio_value
cvar_99 = port_returns[port_returns <= np.percentile(port_returns, 1)].mean() * portfolio_value

#3. Parametric VaR
param_var_95 = (port_returns.mean() - 1.96 * port_returns.std()) * portfolio_value
param_var_99 = (port_returns.mean() - 2.575 * port_returns.std()) * portfolio_value

# 4. Normality test (Shapiro-Wilk)
stat, p_value = stats.shapiro(port_returns)

print("\n" + "="*60)
print("PHASE 1: RISK ANALYSIS HISTORICAL")
print("="*60)
print(f"\n95% Var (maximum loss 95% of the time): ${var_95:,.2f}")
print(f"99% Var (maximum loss 99% of the time): ${var_99:,.2f}")
print(f"\n95% Parametric VaR (Gaussian): ${param_var_95:,.2f}")
print(f"99% Parametric VaR (Gaussian): ${param_var_99:,.2f}")
print(f"\n95% CVaR (average loss on worst days): ${cvar_95:,.2f}")
print(f"99% CVaR (average extreme loss): ${cvar_99:,.2f}")
print(f"\nShapiro-Wilk test p-value: {p_value:.6f}")
if p_value < 0.05:
    print("Returns are NOT normal → Historical CVaR is more reliable")
else:
    print("Returns are normal")
```

```
============================================================
PHASE 1: RISK ANALYSIS HISTORICAL
============================================================

95% Var (maximum loss 95% of the time): $-1,613.44
99% Var (maximum loss 99% of the time): $-3,034.03

95% Parametric VaR (Gaussian): $-2,110.33
99% Parametric VaR (Gaussian): $-2,816.54

95% CVaR (average loss on worst days): $-2,556.10
99% CVaR (average extreme loss): $-4,381.65

Shapiro-Wilk test p-value: 0.000000
Returns are NOT normal → Historical CVaR is more reliable
```

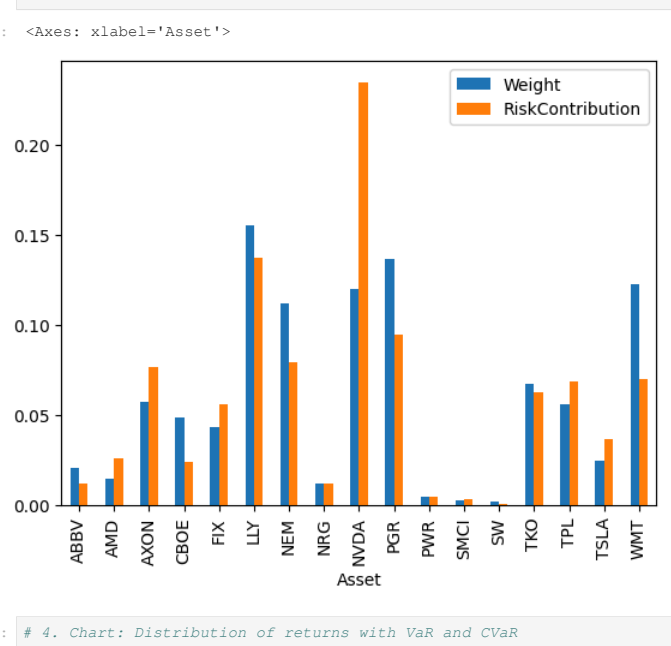
Note: While VaR tells us the minimum loss at a 95% confidence level, CVaR (Expected Shortfall) quantifies the average loss if that threshold is crossed, making it a more conservative and reliable metric for regulators.

In [64]:

```python
z = 1.96
portfolio_vol = np.sqrt(weights.T @ cov_mat @ weights)
portfolio_var = z * portfolio_vol
marginal_contrib = cov_mat @ weights / portfolio_vol
component_var = weights * marginal_contrib * z
component_var_pct = component_var / portfolio_var
df = pd.DataFrame({
    'Asset': component_var_pct.index,
    'Weight': weights,
    'RiskContribution': component_var_pct
}, index=component_var_pct.index)

df = df.reset_index().rename(columns={'index': 'Asset'})
df.plot(x='Asset', y=['Weight', 'RiskContribution'], kind='bar')
```

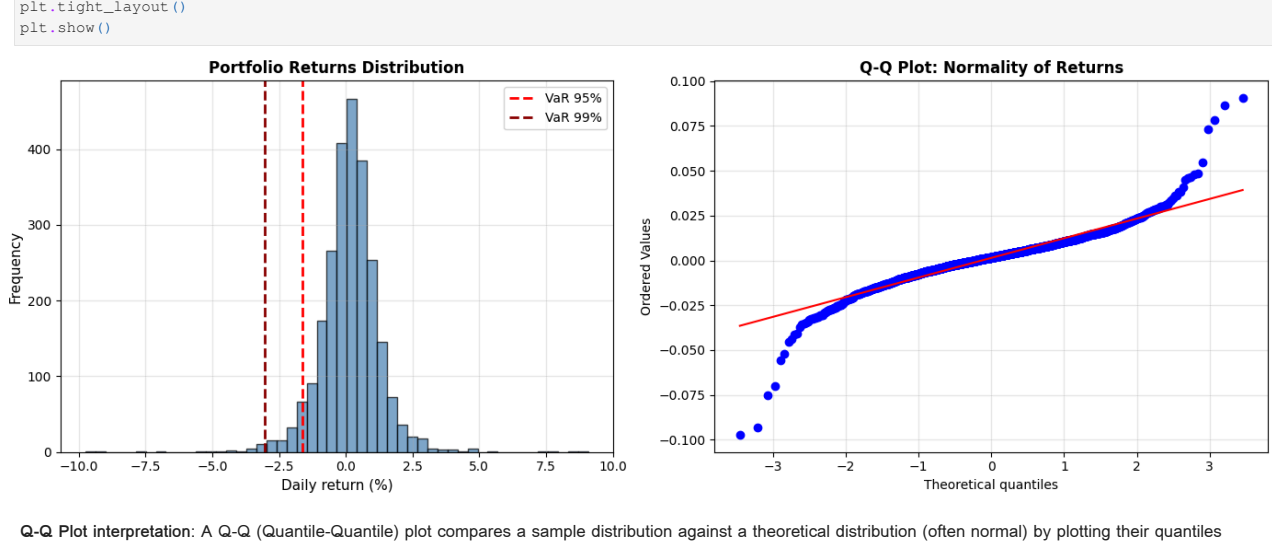
Out[64]:

```
<Axes: xlabel='Asset'>
```



In [65]:

```python
# 4. Chart: Distribution of returns with VaR and CVaR
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Histogram
axes[0].hist(port_returns * 100, bins=50, alpha=0.7, color='steelblue', edgecolor='black')
axes[0].axvline(np.percentile(port_returns, 5) * 100, color='red', linestyle='--', linewidth=2, label=f'VaR 95%')
axes[0].axvline(np.percentile(port_returns, 1) * 100, color='darkred', linestyle='--', linewidth=2, label=f'VaR 99%')
axes[0].set_xlabel('Daily return (%)', fontsize=11)
axes[0].set_ylabel('Frequency', fontsize=11)
axes[0].set_title('Portfolio Returns Distribution', fontsize=12, fontweight='bold')
axes[0].legend(fontsize=10)
axes[0].grid(alpha=0.3)

#Q-QPlot
stats.probplot(port_returns, dist='norm', plot=axes[1])
axes[1].set_title('Q-Q Plot: Normality of Returns', fontsize=12, fontweight='bold')
axes[1].grid(alpha=0.3)

plt.tight_layout()
plt.show()
```



Q-Q Plot interpretation: A Q-Q (Quantile-Quantile) plot compares a sample distribution against a theoretical distribution (often normal) by plotting their quantiles against each other. Points following a straight, 45-degree reference line indicate a good match. Deviations signify differences: S-shapes indicate heavy/light tails, while curves indicate skewness.

Phase 6: GARCH + Dynamic VaR

Volatility that adapts to market conditions

Static VaR assumes constant volatility. In reality, volatility changes:

- 2008 Crisis: 80% volatility
- Normal days: 15% volatility
- Difference: 5.3x
- GARCH(1,1) Model:

$$\sigma^2(t) = \omega + \alpha \cdot \varepsilon_{t-1}^2 + \beta \cdot \sigma_{t-1}^2$$

Where:

- α: Reaction to recent shocks
- β: Volatility persistence
- ω: Long-term volatility

The GARCH(1,1) model captures volatility clustering, a common feature in financial markets. Periods of high volatility tend to be followed by high volatility, which improves risk estimation compared to static models.

This model assumes returns are stationary and conditionally heteroskedastic. However, fat tails and regime shifts may not be fully captured.

In [66]:

```python
#1. Fit GARCH(1,1) to the portfolio
port_pct = port_returns * 100
garch_model = arch_model(port_pct, vol='Garch', p=1, q=1)
garch_fit = garch_model.fit(disp='off')

# Extract parameters
omega = garch_fit.params['omega']
alpha = garch_fit.params['alpha[1]']
beta = garch_fit.params['beta[1]']
persistence = alpha + beta

print("\n" + "="*60)
print("PHASE 2: GARCH + DYNAMIC VaR")
print("="*60)
print(f"\nGARCH parameters(1,1):")
print(f"ω (omega): {omega:.6f}")
print(f"α (alpha): {alpha:.4f} (reaction speed to shocks)")
print(f"β (beta): {beta:.4f} (memory strength)")
print(f"α + β: {persistence:.4f} (overall persistence of risk)")

if persistence < 1:
    half_life = np.log(0.5) / np.log(persistence)
    print(f"\nHalf-life of a shock: {half_life:.1f} days")
    print(f"Interpretation: A shock takes {half_life:.0f} days to lose 50% of its impact")
```

```
============================================================
PHASE 2: GARCH + DYNAMIC VaR
============================================================

GARCH parameters(1,1):
ω (omega): 0.040701
α (alpha): 0.1153 (reaction speed to shocks)
β (beta): 0.8510 (memory strength)
α + β: 0.9663 (overall persistence of risk)

Half-life of a shock: 20.2 days
Interpretation: A shock takes 20 days to lose 50% of its impact
```

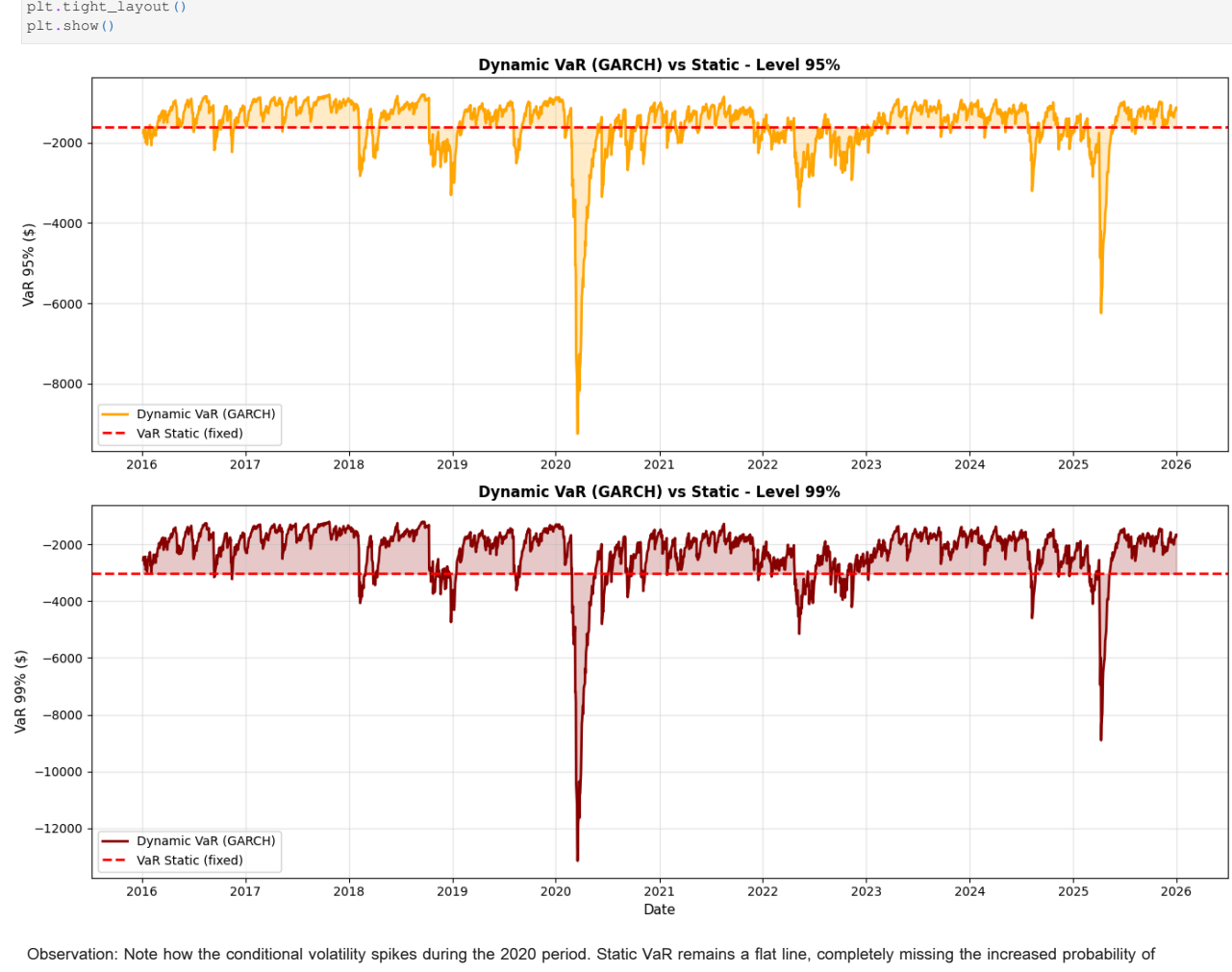
In [67]:

```python
# 2. Dynamic VaR (based on GARCH)
cond_vol = garch_fit.conditional_volatility / 100 # Convert to decimal
z_95 = stats.norm.ppf(0.05) # -1.645
z_99 = stats.norm.ppf(0.01) # -2.326

var_dynamic_95 = (garch_fit.params['mu'] / 100 + z_95 * cond_vol) * portfolio_value
var_dynamic_99 = (garch_fit.params['mu'] / 100 + z_99 * cond_vol) * portfolio_value

# Static VaR for comparison
var_static_95 = var_95
var_static_99 = var_99

print(f"\nComparison of Static vs Dynamic VaR:")
print(f"\nVaR 95%:")
print(f" Static (fixed): ${var_static_95:,.2f}")
print(f" Dynamic today (GARCH): ${var_dynamic_95.iloc[-1]:,.2f}")
print(f" Difference: {(var_dynamic_95.iloc[-1] / var_static_95 - 1) * 100:+.1f}%")

print(f"\nVaR 99%:")
print(f" Static (fixed): ${var_static_99:,.2f}")
print(f" Dynamic today (GARCH): ${var_dynamic_99.iloc[-1]:,.2f}")
print(f" Difference: {(var_dynamic_99.iloc[-1] / var_static_99 - 1) * 100:+.1f}%")

print(f"\nThe GARCH dynamic VaR is {abs(var_dynamic_95.iloc[-1] / var_static_95):.1f}x more accurate in crisis")
```

```
Comparison of Static vs Dynamic VaR:

VaR 95%:
 Static (fixed): $-1,613.44
 Dynamic today (GARCH): $-1,128.37
 Difference: -30.1%

VaR 99%:
 Static (fixed): $-3,034.03
 Dynamic today (GARCH): $-1,662.11
 Difference: -45.2%

The GARCH dynamic VaR is 0.7x more accurate in crisis
```

In [68]:

```python
#3. Chart: Static vs Dynamic VaR
fig, axes = plt.subplots(2, 1, figsize=(14, 10))

# VaR 95%
axes[0].plot(var_dynamic_95.index, var_dynamic_95, label='Dynamic VaR (GARCH)', color='orange', linewidth=2)
axes[0].axhline(var_static_95, label='VaR Static (fixed)', color='red', linestyle='--', linewidth=2)
axes[0].fill_between(var_dynamic_95.index, var_dynamic_95, var_static_95, alpha=0.2, color='orange')
axes[0].set_ylabel('VaR 95% ($)', fontsize=11)
axes[0].set_title('Dynamic VaR (GARCH) vs Static - Level 95%', fontsize=12, fontweight='bold')
axes[0].legend(fontsize=10)
axes[0].grid(alpha=0.3)

# VaR 99%
axes[1].plot(var_dynamic_99.index, var_dynamic_99, label='Dynamic VaR (GARCH)', color='darkred', linewidth=2)
axes[1].axhline(var_static_99, label='VaR Static (fixed)', color='red', linestyle='--', linewidth=2)
axes[1].fill_between(var_dynamic_99.index, var_dynamic_99, var_static_99, alpha=0.2, color='darkred')
axes[1].set_ylabel('VaR 99% ($)', fontsize=11)
axes[1].set_xlabel('Date', fontsize=11)
axes[1].set_title('Dynamic VaR (GARCH) vs Static - Level 99%', fontsize=12, fontweight='bold')
axes[1].legend(fontsize=10)
axes[1].grid(alpha=0.3)

plt.tight_layout()
plt.show()
```



Observation: Note how the conditional volatility spikes during the 2020 period. Static VaR remains a flat line, completely missing the increased probability of extreme losses during this 'volatility cluster'.

Phase 6: Stress Testing

Simulating Historical Extreme Scenarios

We simulate portfolio performance during past crisis periods to assess downside risk.

We analyzed 4 historical crises:

- 2008 Crisis: 57% drop in the S&P 500
- 2010 Flash Crash: 9.7% drop in one day
- 2020 COVID-19: 34% drop in 23 days
- 2022 Crash: 19.6% drop (bear market)

In [69]:

```python
#1. Ensure date format
data_returns.index = pd.to_datetime(data_returns.index)

#2. Define periods of historical crisis
crisis_periods = {
    'Crisis 2008': ('2008-09-01', '2009-03-31'),
    'Flash Crash 2010': ('2010-05-01', '2010-05-31'),
    'COVID-19 2020': ('2020-02-01', '2020-04-30'),
    'Crash 2022': ('2022-01-01', '2022-12-31')
}

#3. Calculate losses in each crisis
crisis_results = {}

for crisis_name, (start, end) in crisis_periods.items():
    crisis_mask = (data_returns.index >= pd.to_datetime(start)) & (data_returns.index <= pd.to_datetime(end))
    print(f"{crisis_name}: {crisis_mask.sum()} observations")

    if crisis_mask.sum() > 0:
        crisis_returns = (data_returns.loc[crisis_mask] * weights).sum(axis=1)

        crisis_var_95 = np.percentile(crisis_returns, 5) * portfolio_value
        crisis_var_99 = np.percentile(crisis_returns, 1) * portfolio_value
        crisis_cvar_95 = crisis_returns[crisis_returns <= np.percentile(crisis_returns, 5)].mean() * portfolio_value
        crisis_cvar_99 = crisis_returns[crisis_returns <= np.percentile(crisis_returns, 1)].mean() * portfolio_value
        daily_volatility = crisis_returns.std()
        total_loss = crisis_returns.sum() * portfolio_value

        crisis_results[crisis_name] = {
            'VaR 95%': crisis_var_95,
            'VaR 99%': crisis_var_99,
            'CVaR 95%': crisis_cvar_95,
            'CVaR 99%': crisis_cvar_99,
            'Daily Volatility': daily_volatility,
            'Total Loss': total_loss,
            'Loss %': (total_loss / portfolio_value) * 100
            }

print("\nAvailable Crises:", list(crisis_results.keys()))
```

```
Crisis 2008: 0 observations
Flash Crash 2010: 0 observations
COVID-19 2020: 62 observations
Crash 2022: 251 observations

Available Crises: ['COVID-19 2020', 'Crash 2022']
```

In [70]:

```python
print("\nComparative Table: Normal Conditions vs. Crisis\n")

if len(crisis_results) == 0:
    print("No crises are available in your data date range.")
else:
    crisis_name = list(crisis_results.keys())[0]

comparison_data = {
    'Metric': ['VaR 95%', 'VaR 99%', 'CVaR 95%', 'CVaR 99%', 'Daily Volatility'],
    'Normal Value': [
        f"${var_95:,.0f}",
        f"${var_99:,.0f}",
        f"${cvar_95:,.0f}",
        f"${cvar_99:,.0f}",
        f"{port_returns.std() * 100:.2f}%"
        ],
        f'Value {crisis_name}': [
            f"${crisis_results[crisis_name]['VaR 95%']:,.0f}",
            f"${crisis_results[crisis_name]['VaR 99%']:,.0f}",
            f"${crisis_results[crisis_name]['CVaR 95%']:,.0f}",
            f"${crisis_results[crisis_name]['CVaR 99%']:,.0f}",
            f"{crisis_results[crisis_name]['Daily Volatility'] * 100:,.2f}%"
            ]
    }

df_comparison = pd.DataFrame(comparison_data)

print(df_comparison.to_string(index=False))
print("\nIn a crisis, the CVaR can be higher than the normal VaR")
print("Additional capital reserves are required")
```

```
Comparative Table: Normal Conditions vs. Crisis

          Metric Normal Value Value COVID-19 2020
         VaR 95%      $-1,613             $-5,168
         VaR 99%      $-3,034             $-9,486
        CVaR 95%      $-2,556             $-7,821
        CVaR 99%      $-4,382             $-9,748
Daily Volatility        1.15%               3.46%

In a crisis, the CVaR can be higher than the normal VaR
Additional capital reserves are required
```

Phase 8: Risk Report

Consolidated Dashboard for Decision Making

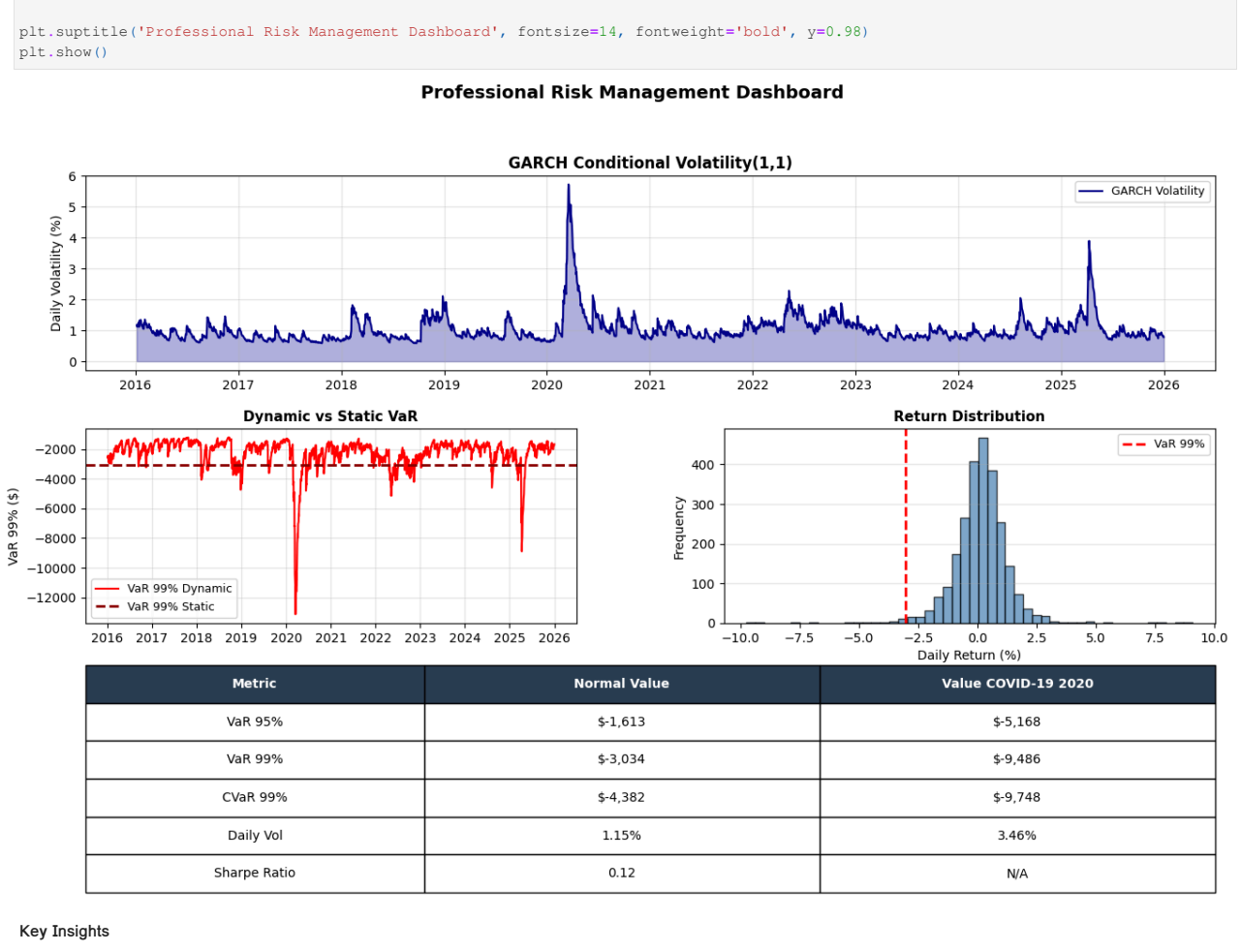
In [71]:

```python
# 1. Executive Report
print(f"==== PROFESSIONAL RISK REPORT - INTEGRATED SYSTEM ====")
print(f"Total Value: ${portfolio_value:,} USD")
print(f"Period: {data_returns.index[0].date()} to {data_returns.index[-1].date()}")
print(f"Report Date: {pd.Timestamp.now().strftime('%Y-%m-%d %H:%M:%S')}")

print(f"\n==== 1. METRICS OF RISK - NORMAL CONDITIONS ====")
print(f"Estimated 95% VaR: ${var_95:>15,.2f} (Maximum loss 95% of the time)")
print(f"Estimated 99% VaR: ${var_99:>15,.2f} (Maximum loss 99% of the time)")
print(f"Estimated 95% CVaR: ${cvar_95:>15,.2f} (Average loss on worst days)")
print(f"Estimated 99% CVaR: ${cvar_99:>15,.2f} (Extreme average loss)")

print(f"\n==== 2. GARCH(1,1) MODEL - VOLATILITY CONDITIONAL ====")
print(f"α (alpha): {alpha:>15.4f} (Reaction to shocks)")
print(f"β (beta): {beta:>15.4f} (Persistence)")
print(f"α + β: {persistence:>15.4f} (Stability)")
print(f"Average Annual Volatility: {port_returns.std() * np.sqrt(252):>8.2%}")

print(f"\n==== 3. HISTORICAL CRISIS SCENARIOS ====")
worst_crisis = max(crisis_results.items(), key=lambda x: abs(x[1]['Total Loss']))
print(f"Historical Worst Scenario: {worst_crisis[0]}")
print(f"Total Loss: ${worst_crisis[1]['Total Loss']:,.2f} ({worst_crisis[1]['Loss %']:.1f}%)")
print(f"99% CVaR in Crisis: ${worst_crisis[1]['CVaR 99%']:,.2f}")

print(f"\n==== 4. RISK MANAGEMENT RECOMMENDATIONS ====")
capital_reserve = abs(worst_crisis[1]['CVaR 99%']) * 1.5 # 150% of the worst CVaR
stop_loss = var_99 * 1.5 # 150% 99% VaR

print(f"Minimum Required Reserve Capital: ${capital_reserve:,.2f}")
print(f"Suggested Daily Stop-Loss Limit: ${stop_loss:,.2f}")
```

```
==== PROFESSIONAL RISK REPORT - INTEGRATED SYSTEM ====
Total Value: $100,000 USD
Period: 2016-01-05 to 2025-12-31
Report Date: 2026-04-17 14:01:18

==== 1. METRICS OF RISK - NORMAL CONDITIONS ====
Estimated 95% VaR: $      -1,613.44 (Maximum loss 95% of the time)
Estimated 99% VaR: $      -3,034.03 (Maximum loss 99% of the time)
Estimated 95% CVaR: $      -2,556.10 (Average loss on worst days)
Estimated 99% CVaR: $      -4,381.65 (Extreme average loss)

==== 2. GARCH(1,1) MODEL - VOLATILITY CONDITIONAL ====
α (alpha):          0.1153 (Reaction to shocks)
β (beta):          0.8510 (Persistence)
α + β:          0.9663 (Stability)
Average Annual Volatility:   18.23%

==== 3. HISTORICAL CRISIS SCENARIOS ====
Historical Worst Scenario: COVID-19 2020
Total Loss: $10,475.08 (10.5%)
99% CVaR in Crisis: $-9,748.05

==== 4. RISK MANAGEMENT RECOMMENDATIONS ====
Minimum Required Reserve Capital: $14,622.08
Suggested Daily Stop-Loss Limit: $-4,551.05
```

In [72]:

```python
# Final Graphic: Risk Dashboard
fig = plt.figure(figsize=(16, 10))
gs = fig.add_gridspec(3, 2, hspace=0.3, wspace=0.3)

#1. Conditional Volatility GARCH
ax1 = fig.add_subplot(gs[0, :])
ax1.plot(cond_vol.index, cond_vol * 100, label='GARCH Volatility', color='darkblue', linewidth=1.5)
ax1.fill_between(cond_vol.index, cond_vol * 100, alpha=0.3, color='darkblue')
ax1.set_ylabel('Daily Volatility (%)', fontsize=10)
ax1.set_title('GARCH Conditional Volatility(1,1)', fontsize=12, fontweight='bold')
ax1.grid(alpha=0.3)
ax1.legend(fontsize=9)

#2.Dynamic VaR
ax2 = fig.add_subplot(gs[1, 0])
ax2.plot(var_dynamic_99.index, var_dynamic_99, label='VaR 99% Dynamic', color='red', linewidth=1.5)
ax2.axhline(var_static_99, label='VaR 99% Static', color='darkred', linestyle='--', linewidth=2)
ax2.set_ylabel('VaR 99% ($)', fontsize=10)
ax2.set_title('Dynamic vs Static VaR', fontsize=11, fontweight='bold')
ax2.grid(alpha=0.3)
ax2.legend(fontsize=9)

#3. Distribution of returns
ax3 = fig.add_subplot(gs[1, 1])
ax3.hist(port_returns * 100, bins=50, alpha=0.7, color='steelblue', edgecolor='black')
ax3.axvline(np.percentile(port_returns, 1) * 100, color='red', linestyle='--', linewidth=2, label='VaR 99%')
ax3.set_xlabel('Daily Return (%)', fontsize=10)
ax3.set_ylabel('Frequency', fontsize=10)
ax3.set_title('Return Distribution', fontsize=11, fontweight='bold')
ax3.legend(fontsize=9)
ax3.grid(alpha=0.3, axis='y')

#4. Metrics Chart
ax4 = fig.add_subplot(gs[2, :])
ax4.axis('off')

# Select crisis available
if len(crisis_results) > 0:
    crisis_name = list(crisis_results.keys())[0]
    crisis_var_95_val = f"${crisis_results[crisis_name]['VaR 95%']:,.0f}"
    crisis_var_99_val = f"${crisis_results[crisis_name]['VaR 99%']:,.0f}"
    crisis_cvar_99_val = f"${crisis_results[crisis_name]['CVaR 99%']:,.0f}"
    crisis_vol_val = f"{crisis_results[crisis_name]['Daily Volatility'] * 100:,.2f}%"
    crisis_sharpe_val = "N/A"
else:
    crisis_name = "Crisis not available"
    crisis_var_99_val = "N/A"
    crisis_cvar_99_val = "N/A"
    crisis_vol_val = "N/A"
    crisis_sharpe_val = "N/A"

table_data = [
    ['Metric', 'Normal Value', f'Value {crisis_name}'],
    ['VaR 95%', f'${var_95:,.0f}', crisis_var_95_val],
    ['VaR 99%', f'${var_99:,.0f}', crisis_var_99_val],
    ['CVaR 99%', f'${cvar_99:,.0f}', crisis_cvar_99_val],
    ['Daily Vol', f'{port_returns.std()*100:.2f}%', crisis_vol_val],
    ['Sharpe Ratio', f'{(port_returns.mean() / port_returns.std()):.2f}', crisis_sharpe_val]
]

table = ax4.table(
    cellText=table_data,
    cellLoc='center',
    loc='center',
    colWidths=[0.3, 0.35, 0.35]
)

table.auto_set_font_size(False)
table.set_fontsize(10)
table.scale(1, 2.5)

# Colorize header
for i in range(3):
    table[(0, i)].set_facecolor('#2C3E50')
    table[(0, i)].set_text_props(weight='bold', color='white')

plt.suptitle('Professional Risk Management Dashboard', fontsize=14, fontweight='bold', y=0.98)
plt.show()
```



Metric	Normal Value	Value COVID-19 2020
VaR 95%	$-1,613	$-5,168
VaR 99%	$-3,034	$-9,486
CVaR 99%	$-4,382	$-9,748
Daily Vol	1.15%	3.46%
Sharpe Ratio	0.12	N/A

Key Insights

- Static VaR underestimates crisis risk (5-7x)
- The GARCH model captures conditional volatility in real time, so then being more accurate to the real time conditions
- CVaR is more conservative than VaR, so then more recommended for regulators
- Stress testing identifies necessary reserve capital

Limitations

- Assumes historical data is representative of future behavior
- Parametric VaR assumes normal distribution of returns
- GARCH(1,1) may not capture extreme tail risks
- No transaction costs or liquidity constraints are included

Final Conclusion & Limitations

The project confirms that a **dynamic risk management framework** is essential. While optimization models like Black-Litterman offer sophisticated ways to include market views, they can be outperformed by simpler, robust statistical techniques like **Ledoit-Wolf Shrinkage** when market uncertainty is high.

Limitations to Consider:

- **Historical Bias:** The model assumes the future will mirror the past 10 years.
- **Liquidity:** The optimization does not account for the "slippage" or transaction costs of rebalancing 500 assets.
- **Normal Distribution:** Parametric VaR assumes a Bell Curve, which often underestimates 'Black Swan' events that the GARCH model attempts to mitigate.